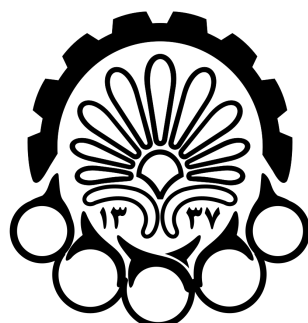




دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

دانشکده مهندسی کامپیوتر

پایان نامه پروژه کارشناسی

عنوان پروژه:

کاهش هشدارهای نادرست ابزارهای تحلیل ایستا
به کمک بازیابی مبتنی بر شواهد با عامل زبانی

نگارنده:

امیرحسین قدیری

استاد راهنما:

مهران سلیمان فلاح

تابستان ۱۴۰۴

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

اینجانب امیرحسین قدیری متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آن‌ها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه صنعتی امیرکبیر است. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است.

نقل مطالب با ذکر مآخذ بلامانع است.

امیرحسین قدیری

امضا



سپاس‌گزاری

بدینوسیله مراتب قدردانی و امتنان خود را خدمت جناب آقای دکتر سلیمان فلاح، استاد راهنمای گرانقدر، که راهنمایی‌های خود را در تمام مراحل بر من ارزانی داشتند و بدون کمک‌های ایشان انجام این پروژه امکان‌پذیر نبود، ابراز و از زحمات ایشان بسیار سپاس‌گزار بوده و آرزومند توفیق روزافزون برایشان می‌باشم. همچنین از خانواده و دوستانی که با صبوری و همراهی خود در طول انجام این پژوهش مرا یاری رساندند سپاسگزارم. این پروژه بدون بهره‌گیری از ابزارهای هوش مصنوعی مولد و مدل‌های زبانی بزرگ، از جمله سامانه‌های گفت‌وگویی مبتنی بر مدل، جهت طراحی، پیاده‌سازی و ارزیابی سامانه ممکن نبود و از این بابت سپاس و قدردانی می‌کنم.

تابستان ۱۴۰۴

تقدیم

تقدیم به پدر و مادر عزیزم، که با مهربانی، صبر و حمایت بی‌دریغ خود همواره پشتوانه و چراغ راه من بوده‌اند. محبت و انگیزه‌ای که از شما دریافت کرده‌ام، همواره مشوق من در مسیر تحصیل و تلاش علمی بوده و بدون همراهی و دلگرمی‌های شما، دستیابی به این مرحله امکان‌پذیر نبوده است.

تابستان ۱۴۰۴

چکیده

تحلیل ایستا یکی از پرکاربردترین روش‌های خودکار برای کشف آسیب‌پذیری‌های امنیتی در نرم‌افزار است. با این حال، این ابزارها با نرخ بالای هشدارهای نادرست همراه‌اند؛ هشدارهایی که به محل‌هایی از کد اشاره می‌کنند که در واقع ایرادی ندارند. این مسئله سبب کاهش اعتماد توسعه‌دهندگان و در نهایت کنار گذاشتن ابزار می‌شود. از سوی دیگر، مدل‌های زبانی بزرگ در درک زمینه اجرای برنامه توانمندند، اما به‌تنهایی نمی‌توانند به‌عنوان داور قابل‌اتکای یافته‌های امنیتی به کار روند.

در این پژوهش، سامانه‌ای سه‌فاز برای کاهش هشدارهای نادرست تحلیل ایستا به کمک بازبینی مبتنی بر شواهد با عامل زبانی طراحی و پیاده‌سازی شد. فاز نخست، سه ابزار تحلیل ایستا را به‌صورت موازی روی هر فایل اجرا می‌کند. فاز دوم، یافته‌های تکراری را با استفاده از اثر انگشت و تطبیق خط در گروه‌هایی که هر یک نماینده یک نقطه ضعف واحد هستند، ادغام می‌کند. فاز سوم، هر گروه را در قالب بسته‌ای ساخت‌یافته شامل قطعه کد، پیام ابزارها و شواهد ساختاری به عامل زبانی ارائه می‌دهد و از او یکی از سه تصمیم تأیید، رد یا عدم قطعیت را می‌گیرد. برای آنکه نتیجه ارزیابی بازتاب توان واقعی سامانه باشد، ماژولی برای پاک‌سازی کد پیاده‌سازی شد که برچسب واقعی هر فایل را که در متن فایل و نام اشیا نهفته است، بدون تغییر شماره خط گزارش‌شده توسط ابزارها، از دید مدل پنهان می‌کند.

ارزیابی تجربی روی زیرمجموعه‌ای متعادل و لایه‌بندی‌شده از مجموعه داده جولیت شامل ۶۰۰ فایل، شامل ۳۰۰ فایل آسیب‌پذیر و ۳۰۰ فایل سالم، و در دو نوع آسیب‌پذیری سرریز بافر و سرریز عدد صحیح انجام شد. چهار پیکربندی شامل تحلیل ایستای تنها، عامل زبانی تنها، ترکیب تحلیل ایستا و عامل، و ترکیب تحلیل ایستا با شواهد ساختاری و عامل، روی یک جمعیت یکسان مقایسه شدند.

نتایج نشان می‌دهد که بازبینی مبتنی بر شواهد، نرخ هشدار روی فایل‌های سالم را از ۰/۰۵۰ به ۰/۰۰۷، معادل حدود ۸۷ درصد کاهش، رساند و دقت یافته‌ها را از ۰/۳۷۹ به ۰/۶۰۰ افزایش داد؛ اما یادآوری از ۰/۰۳۷ به ۰/۰۱۰ افت کرد. افزودن شواهد ساختاری، یادآوری را به ۰/۰۱۷ رساند و تعداد تأییدهای عامل را بیش از دو برابر کرد. همچنین مشخص شد که هیچ‌یک از پیکربندی‌های مبتنی بر تحلیل ایستا، حتی یک نمونه از ۱۵۰ نمونه سرریز عدد صحیح را کشف نکردند، در حالی که عامل زبانی به‌تنهایی در این نوع آسیب‌پذیری به یادآوری ۰/۰۶۰ رسید. این نتایج نشان می‌دهد عامل زبانی، ابزار مؤثری برای پالایش هشدارهای تحلیل ایستا است، اما بهره آن به کیفیت و دانه‌بندی شواهد ارائه‌شده وابسته است.

واژه‌های کلیدی:

تحليل ایستا، هشدار نادرست، عامل زبانی، مدل زبانی بزرگ، آسیب‌پذیری نرم‌افزاری، مجموعه داده جولیت، بازبینی مبتنی بر شواهد

فهرست مطالب

فصل اول - مقدمه	۱
۱.۱	۱.۱
بیان مسئله	۲
۱.۲	۱.۲
اهمیت و ضرورت پژوهش	۳
۱.۳	۱.۳
اهداف پژوهش	۴
۱.۴	۱.۴
دستاوردهای پژوهش	۵
۱.۵	۱.۵
روش پیشنهادی	۵
۱.۶	۱.۶
ساختار پایان نامه	۶
فصل دوم - مبانی نظری و کارهای پیشین	۷
۲.۱	۲.۱
تحلیل ایستا و مفهوم هشدار نادرست	۸
۲.۲	۲.۲
ابزارهای تحلیل ایستای به کاررفته در این پژوهش	۸
۲.۳	۲.۳
تحلیل ایستا-پویا و رویکردهای ترکیبی	۹
۲.۴	۲.۴
مدل های زبانی بزرگ و عامل های نرم افزاری	۹
۲.۵	۲.۵
استدلال مبتنی بر شواهد	۱۰
۲.۶	۲.۶
مجموعه داده جولیت	۱۰

۲.۷

کارهای پیشین ۱۱

۲.۸

جایگاه کار پیشنهادی ۱۲

فصل سوم - طراحی و پیاده‌سازی سامانه ۱۳

۳.۱

معماری کلی ۱۴

۳.۲

فاز یک: تحلیل ایستا ۱۴

۳.۳

فاز دو: نرمال‌سازی و همبسته‌سازی یافته‌ها ۱۴

۳.۴

فاز سه: بازبینی مبتنی بر شواهد ۱۵

۳.۵

بسته شواهد ۱۵

۳.۶

پاک‌سازی کد و جلوگیری از نشت برچسب ۱۶

۳.۷

سیاست تطبیق یافته‌ها و معیارها ۱۷

۳.۸

زیرساخت اجرا و تکرارپذیری ۱۸

فصل چهارم - ارزیابی تجربی ۱۹

۴.۱

پرسش‌های پژوهش ۲۰

۴.۲

مجموعه داده و آماده‌سازی آن ۲۰

۴.۳

طراحی آزمایش‌ها ۲۰

۴.۴	
۲۱	معیارها و سیاست تطبیق
۴.۵	
۲۱	نتایج اصلی
۴.۶	
۲۲	تحلیل قیف تصمیم
۴.۷	
۲۳	نتایج به تفکیک نوع آسیب‌پذیری
۴.۸	
۲۴	بررسی در مقیاس کامل
۴.۹	
۲۴	تهدیدهای روایی
۲۶	فصل پنجم - نتیجه‌گیری و کارهای آینده
۵.۱	
۲۷	جمع‌بندی
۵.۲	
۲۷	محدودیت‌ها
۵.۳	
۲۷	کارهای آینده
۲۹	فهرست مراجع

فهرست شکل‌ها

شکل ۱.۱

-

نمونه‌ای از کد آسیب‌پذیر از نوع سرریز عدد صحیح ۳

شکل ۱.۲

-

نسخه اصلاح‌شده همان سناریو با بررسی پیش از محاسبه ۳

شکل ۱.۳

-

نمای کلی سامانه پیشنهادی و جایگاه بازیابی توسط عامل زبانی ۴

شکل ۲.۱

-

نسخه آسیب‌پذیر یک سناریوی جولیت ۱۰

شکل ۲.۲

-

نسخه سالم همان سناریو؛ تنها تفاوت، یک شرط کنترلی است ۱۱

شکل ۳.۱

-

معماری سه‌فاز سامانه و جریان داده میان فازها ۱۴

شکل ۳.۲

-

ساختار بسته شواهد ارسال‌شده به عامل زبانی ۱۶

شکل ۳.۳

-

قطعه کد پس از پاک‌سازی؛ حاشیه‌نویسی‌ها محو و شناسه‌های افشاگر بازنام‌گذاری شده‌اند ۱۷

شکل ۳.۴

-

زنجیره ادغام یافته‌های تکراری و تشکیل گروه‌های بازبینی ۱۵

شکل ۴.۱

-

مقایسه دقت، یادآوری و نرخ هشدار روی فایل سالم در چهار پیکربندی ۲۱

شکل ۴.۲

-

ماتریس سردرگمی چهار پیکربندی در دانه‌بندی فایل و یافته ۲۲

شکل ۴.۳

-

اثر شواهد ساختاری بر تصمیم‌های عامل زبانی ۲۳

فهرست جدول‌ها

جدول ۲.۱

-

ویژگی ابزارهای تحلیل ایستای به‌کاررفته در فاز یک ۸

جدول ۲.۲

-

جایگاه کار پیشنهادی در میان کارهای پیشین ۱۱

جدول ۳.۱

-

سه خروجی ممکن عامل زبانی ۱۵

جدول ۳.۲

-

سیاست تطبیق یافته‌ها با برچسب واقعی ۱۸

جدول ۴.۱

-

مشخصات مجموعه داده ارزیابی و پیکربندی آزمایش ۲۰

جدول ۴.۲

-

نتایج اصلی چهار آزمایش روی ۶۰۰ نمونه متعادل ۲۱

جدول ۴.۳

-

قیف تصمیم عامل زبانی در هر پیکربندی ۲۳

جدول ۴.۴

-

نتایج به تفکیک نوع آسیب‌پذیری ۲۴

فصل اول

مقدمه

۱.۱ بیان مسئله

تحلیل ایستا^۱ یکی از پرکاربردترین رویکردها برای یافتن آسیب‌پذیری‌های امنیتی در نرم‌افزار است. در این رویکرد، کد پروژه بدون اجرا شدن و با استفاده از تحلیل نحوی، کنترل جریان داده و مجموعه‌ای از قواعد از پیش تعریف‌شده بررسی می‌شود و ابزار، محل‌هایی از کد را که ممکن است دربردارنده خطا باشند به‌عنوان هشدار گزارش می‌کند. مزیت اصلی این رویکرد آن است که نیازی به اجرای برنامه یا آماده‌سازی ورودی‌های خاص نبوده و در نتیجه می‌توان آن را در مراحل ابتدایی چرخه توسعه و بر روی کل مخزن نرم‌افزاری به کار برد. به همین دلیل، ابزارهایی مانند [۱] cppcheck و [۲] flawfinder در فرآیندهای یکپارچه ساخت و زنجیره‌های ابزار توسعه جایگاه تثبیت‌شده‌ای پیدا کرده‌اند.

با این حال، مهم‌ترین چالش عملی ابزارهای تحلیل ایستا، نرخ بالای هشدارهای نادرست^۲ است. هشدار نادرست یعنی ابزار محلی را به‌عنوان محل آسیب‌پذیری معرفی کرده است، در حالی که آن مسیر در واقع اجرا نمی‌شود، یا شرط لازم برای بهره‌برداری مهاجم هرگز برقرار نمی‌شود. بررسی‌های انجام‌شده روی پروژه‌های واقعی نشان می‌دهد که سهم قابل‌توجهی از هشدارهای این ابزارها در عمل نادرست است و همین موضوع سبب می‌شود توسعه‌دهندگان به‌تدریج به هشدارها بی‌اعتماد شوند و در نهایت ابزار را کنار بگذارند. بنابراین مسئله اصلی، افزایش تعداد هشدارها نیست، بلکه افزایش نسبت هشدارهای مفید و کاهش هشدارهای نادرست در عین حفظ توانایی کشف آسیب‌پذیری‌های واقعی است. در سال‌های اخیر، مدل‌های زبانی بزرگ^۳ توانسته‌اند درک معنایی از کد به دست آورند و این امکان را ایجاد کرده‌اند که بخشی از قضاوت‌های نیازمند فهم زمینه اجرای برنامه، به‌جای قواعد صرفاً نحوی، به این مدل‌ها واگذار شود. برای نمونه، بازبینی هشدار یک ابزار ایستا توسط یک مدل زبانی می‌تواند بررسی کند که آیا مسیر داده واقعاً قابل دسترس است، آیا مقدار مورد استفاده پیش از رسیدن به نقطه خطر اعتبارسنجی می‌شود و آیا نام توابع فراخوانی‌شده با الگوهای شناخته‌شده خطرناک همخوانی دارد یا خیر. با این حال، اتکال صرف به مدل زبانی نیز ریسک‌هایی به همراه دارد. این مدل‌ها ممکن است یافته‌ای را بدون شاهد کافی تأیید کنند، دلایل نادرست تولید کنند و در صورت مواجهه با کد مبهم، به‌جای اعلام عدم قطعیت، پاسخ قطعی تولید کنند. به همین دلیل، استفاده از آن‌ها به‌عنوان جایگزین کامل ابزارهای تحلیل ایستا، دست‌کم در ارزیابی‌های کمی، نتیجه رضایت‌بخشی به همراه نداشته است.

مسئله‌ای که این پژوهش بر آن تمرکز دارد، طراحی و ارزیابی یک سامانه ترکیبی است که در آن تحلیل ایستا نقش پیشنهاددهنده نامزدها را بر عهده می‌گیرد و یک عامل زبانی نقش داور مبتنی بر شواهد را ایفا می‌کند. هدف این سامانه آن است که هشدارهای نادرست ابزارهای ایستا را بدون از دست دادن آسیب‌پذیری‌های واقعی کاهش دهد. برای آنکه ادعای کاهش هشدار نادرست قابل دفاع باشد، لازم است ابتدا مشخص شود که معیار «کاهش» دقیقاً بر چه پایه‌ای سنجیده می‌شود. این موضوع در فصل چهارم با معرفی سیاست تطبیق یافته‌ها و تفکیک دانه‌بندی معیارها بررسی

^۱ Static Analysis

^۲ False Positive

^۳ Large Language Model

می‌شود، اما به‌طور خلاصه، در این پایان‌نامه دو دانه‌بندی متمایز گزارش می‌شود: دانه‌بندی یافته که تطابق میان هشدار ابزار و محل واقعی خطا را می‌سنجد، و دانه‌بندی فایل که میزان آلودگی هر فایل سالم به هشدار را اندازه می‌گیرد. تنها تعریف دقیق این دو دانه‌بندی است که اجازه می‌دهد درباره کاهش هشدار نادرست سخن گفته شود، زیرا در غیر این صورت، کاهش تعداد هشدارها می‌تواند صرفاً نتیجه حذف تصادفی یافته‌ها باشد.

CWE190_Integer_Overflow__int_rand_add_73b_bad.cpp

```
void badSink(list<int> dataList)
{
    int data = dataList.back();
    {
        /* POTENTIAL FLAW: Adding 1 to data could
           cause an overflow */
        int result = data + 1;
        printIntLine(result);
    }
}
```

شکل ۱.۱: نمونه‌ای از کد آسیب‌پذیر از نوع سرریز عدد صحیح

CWE190_Integer_Overflow__int_rand_add_73b_good.cpp

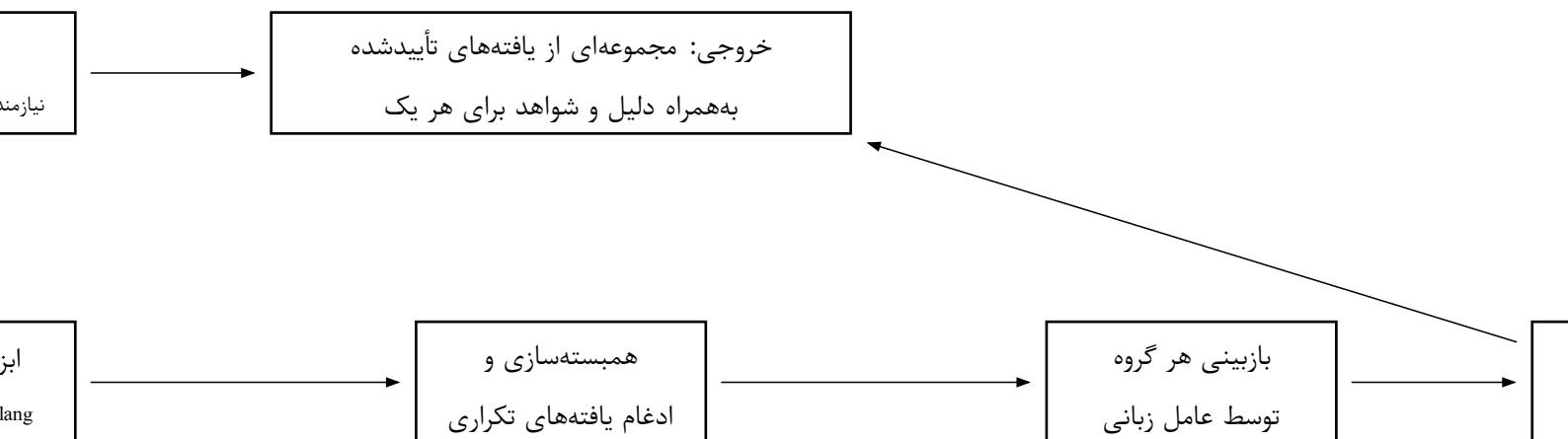
```
void goodB2GSink(list<int> dataList)
{
    int data = dataList.back();
    if (data < INT_MAX) {
        int result = data + 1;
        printIntLine(result);
    } else {
        printLine("data value is too large ...");
    }
}
```

شکل ۱.۲: نسخه اصلاح‌شده همان سناریو با بررسی پیش از محاسبه

۱.۲ اهمیت و ضرورت پژوهش

آسیب‌پذیری‌های ایمنی حافظه، از جمله سرریز بافر روی پشته و سرریز بافر مبتنی بر حافظه چرخه‌ای^۴ و سرریز عدد صحیح^۵، همچنان از پرتکرارترین و خطرناک‌ترین دسته‌های آسیب‌پذیری در نرم‌افزارهای نوشته‌شده به زبان‌های C و C++ هستند. سازوکار این آسیب‌پذیری‌ها به‌گونه‌ای است که بسیاری از آن‌ها تنها با دنبال کردن جریان داده از منبع تا نقطه خطر و بررسی شرط‌های کنترل‌کننده آشکار می‌شوند؛ یعنی همان تحلیلی که ابزارهای ایستا انجام می‌دهند و به همین دلیل نیز بیشترین حجم هشدارهای نادرست از همین دسته تولید می‌شود. این تنش، یعنی ضرورت پوشش آسیب‌پذیری در کنار پرهیز از هشدار نادرست، مسئله مرکزی این پژوهش است.

اهمیت موضوع زمانی بیشتر می‌شود که هزینه بررسی دستی یک هشدار نادرست در نظر گرفته شود. هر هشدار نادرست، زمان توسعه‌دهنده را صرف خواندن کدی می‌کند که در نهایت سالم ارزیابی می‌شود و همین هزینه، انگیزه استفاده از ابزار را از بین می‌برد. در مقابل، یک آسیب‌پذیری که دیده نشود، ممکن است ماه‌ها یا سال‌ها در سامانه باقی بماند. بنابراین سامانه‌ای که بتواند با هزینه محاسباتی اندک، سیگنال اولیه ابزارهای ایستا را پالایش کند و تنها موارد دارای شاهد کافی را به توسعه‌دهنده نشان دهد، ارزش عملی روشی دارد.



شکل ۱.۳: نمای کلی سامانه پیشنهادی و جایگاه بازبینی توسط عامل زبانی

۱.۳ اهداف پژوهش

- هدف کلی این پژوهش، طراحی، پیاده‌سازی و ارزیابی کمی یک سامانه ترکیبی برای کاهش هشدارهای نادرست تحلیل ایستا در کد C و C++ به کمک بازبینی مبتنی بر شواهد با عامل زبانی است. اهداف جزئی به شرح زیر بیان می‌شوند:
- طراحی معماری سه‌فاز شامل فاز تحلیل ایستا، فاز ادغام و همبسته‌سازی یافته‌ها و فاز بازبینی با عامل زبانی، به‌گونه‌ای که مرز مسئولیت هر فاز روشن و قابل آزمون باشد.
 - تعریف بسته شواهد ساخت‌یافته شامل قطعه کد پاک‌سازی‌شده، پیام ابزارها و شواهد ساختاری، به‌منظور آنکه تصمیم عامل زبانی بر پایه شواهد و نه بر پایه حدس گرفته شود.

^۴Heap-based Buffer Overflow

^۵Integer Overflow

- پیاده‌سازی سازوکاری برای جلوگیری از نشت برچسب^۶ در مجموعه داده ارزیابی، به گونه‌ای که نتیجه آزمایش‌ها بازتاب توانایی واقعی سامانه باشد و نه توانایی خواندن پاسخ از درون فایل آزمون.
- تدوین چارچوب ارزیابی تکرارپذیر شامل سیاست تطبیق یافته‌ها، تفکیک دانه‌بندی معیارها و گزارش نتایج به تفکیک نوع آسیب‌پذیری.
- بررسی تجربی اثر هر جزء سامانه از طریق طراحی چهار آزمایش مقایسه‌ای و تحلیل نتایج آن‌ها.

۱.۴ دستاوردهای پژوهش

- دستاوردهای اصلی این پژوهش به شرح زیر است:
- پیاده‌سازی یک سامانه سه‌فاز به صورت یک بسته نرم‌افزاری قابل اجرا، شامل اجرای موازی چند ابزار تحلیل ایستا، ادغام و همبسته‌سازی یافته‌ها و بازبینی هر گروه یافته توسط یک عامل زبانی.
 - ارائه ماژول پاک‌سازی کد که با محوکردن حاشیه‌نویسی‌ها در جای خود و بازنام‌گذاری شناسه‌های افشاگر، برچسب واقعی را از دید مدل پنهان می‌کند و در عین حال شماره خط گزارش شده توسط ابزارها را تغییر نمی‌دهد.
 - ارائه چارچوب ارزیابی تکرارپذیر شامل چهار آزمایش مقایسه‌ای، سازوکار تطبیق گران‌خوار، معیارهای دانه‌بندی یافته و دانه‌بندی فایل و گزارش خودکار جدول مقایسه.
 - به‌کارگیری مجموعه داده جولیت به عنوان مجموعه ارزیابی و گزارش نتایج آن بر روی مجموعه‌ای متعادل و لایه‌بندی شده از نظر نوع آسیب‌پذیری و برچسب.
 - تحلیل نتایج و استخراج این نتیجه‌گیری که اثربخشی بازبینی مبتنی بر شواهد به کیفیت و دانه‌بندی شواهد ارائه شده وابسته است.

۱.۵ روش پیشنهادی

روش پیشنهادی این پژوهش، بر پایه این مشاهده استوار است که تحلیل ایستا و مدل زبانی، نقطه قوت مکمل دارند. ابزار ایستا نامزدهای بی‌شمار و ارزان تولید می‌کند، اما در قضاوت نهایی ضعیف است؛ مدل زبانی در فهم زمینه اجرا توانمند است، اما به تنهایی نامزد قابل اتکایی پیش نمی‌دهد. بر این اساس، در سامانه پیشنهادی، ابزارهای ایستا تنها به عنوان تأمین‌کننده نامزد به کار می‌روند و تصمیم نهایی بر عهده یک عامل زبانی است که موظف است تنها بر پایه بسته شواهد ارائه شده داوری کند.

برای آنکه داوری عامل زبانی قابل اعتماد باشد، سه تصمیم طراحی اتخاذ شده است. نخست، شواهد به صورت ساخت یافته و با کمترین زمینه اضافی ارائه می‌شود تا هم حجم توکن کاهش یابد و هم تمرکز مدل بر همان نقطه مورد ادعا بماند. دوم، هر متنی که ممکن است برچسب را افشا کند، پیش از ارسال پاک‌سازی می‌شود. سوم، به جای دوگزینه‌ای بودن، سه خروجی تعریف شده است تا عامل بتواند در نبود شاهد کافی، صریحاً عدم قطعیت خود را اعلام کند.

^۶Label Leakage

۱.۶ ساختار پایان‌نامه

ساختار این پایان‌نامه به شرح زیر است. در فصل دوم، مبانی نظری شامل تحلیل ایستا، تحلیل ایستا-پویا، مدل‌های زبانی بزرگ و عامل‌های نرم‌افزاری، مجموعه داده جولیت و کارهای پیشین بررسی می‌شود. در فصل سوم، معماری سامانه، ساختار بسته شواهد، سازوکار پاک‌سازی کد، سیاست تطبیق یافته‌ها و معیارهای ارزیابی تشریح می‌شود. در فصل چهارم، طراحی آزمایش‌ها، نتایج به دست آمده، تحلیل به‌ازای نوع آسیب‌پذیری، بررسی نمونه‌های خطا و تهدیدهای روایی ارائه می‌شود. در فصل پنجم، نتیجه‌گیری، محدودیت‌ها و کارهای آینده بیان می‌شود.

فصل دوم

مبانی نظری و کارهای پیشین

۲.۱ تحلیل ایستا و مفهوم هشدار نادرست

تحلیل ایستا مجموعه‌ای از روش‌های خودکار است که هدف آن، یافتن خطاها و نقض‌های قرارداد در برنامه بدون اجرای آن است. این روش‌ها بر پایه ساخت یک نمایش میانی از برنامه، مانند گراف کنترل جریان یا درخت نحو انتزاعی، و سپس اعمال قواعد تحلیل روی آن نمایش استوارند. دو خانواده اصلی از تحلیل ایستا عبارت‌اند از: تحلیل مبتنی بر قواعد^۱ که با الگوهای از پیش تعریف‌شده الگوهای بد، مانند استفاده از فراخوانی‌های خطرناک، کار می‌کند؛ و تحلیل مبتنی بر تفسیر انتزاعی^۲ که با تقریب زدن مجموعه مقادیر ممکن یک متغیر در هر نقطه برنامه، به صورت ایستا به رفتار برنامه نزدیک می‌شود. این دو خانواده، مبنای تقریباً همه ابزارهای تجاری و متن‌باز امروزی هستند.

هر ابزار تحلیل ایستا زمانی ارزشمند است که بتواند میان سه دسته هشدار تمایز بگذارد: هشدار درست، هشدار نادرست و از دست رفتن آسیب‌پذیری واقعی. هشدار درست یعنی محل گزارش‌شده واقعاً یک نقص امنیتی دارد. هشدار نادرست یعنی محل گزارش‌شده ایرادی ندارد؛ برای نمونه مسیر داده هشدار داده‌شده هرگز اجرا نمی‌شود، یا اندازه بافر پیش از استفاده کنترل شده است. از دست رفتن آسیب‌پذیری واقعی یعنی ابزار ساکت مانده است، در حالی که کد آسیب‌پذیر است. اندازه‌گیری صرفاً دقت و یادآوری، بدون توجه به این‌که این معیارها با چه دانه‌بندی و با چه سیاستی محاسبه شده‌اند، مقایسه ابزارها را گمراه‌کننده می‌کند؛ از این رو در فصل چهارم دانه‌بندی معیارها به صراحت تعریف و در همه آزمایش‌ها به طور یکسان اعمال می‌شود.

بررسی‌های پیشین نشان می‌دهد که نرخ هشدار نادرست در ابزارهای تحلیل ایستای C و C++ بالا است و در برخی مطالعات به بیش از نیمی از کل هشدارهای گزارش‌شده رسیده است [۶] [۷] [۸]. همچنین، بررسی‌های میدانی از توسعه‌دهندگان نشان داده است که حتی توسعه‌دهندگان باتجربه نیز در شناسایی مسائل امنیتی کد دچار خطا می‌شوند و هشدارهای متناقض ابزارها اعتماد آنان را کاهش می‌دهد [۶].

۲.۲ ابزارهای تحلیل ایستای به‌کاررفته در این پژوهش

در فاز یک سامانه پیشنهادی، سه ابزار متن‌باز به صورت موازی روی هر فایل اجرا می‌شوند. انتخاب این سه ابزار بر پایه تنوع رویکرد آن‌ها انجام شده است: یکی مبتنی بر قواعد ساده و سریع، یکی مبتنی بر تحلیل جریان داده و درونیابی، و یکی مبتنی بر تحلیل درخت نحو انتزاعی.

ابزار	رویکرد	نقطه قوت	نقطه ضعف
cppcheck	قاعده‌محور	سریع و کم‌هزینه	عمق تحلیل کم
flawfinder	قاعده‌محور	پوشش سبک‌های خطرناک	بسیار پرهشدار

^۱ Rule-based

^۲ Abstract Interpretation

ابزار	رویکرد	نقطه قوت	نقطه ضعف
clang	درخت نحو انتزاعی	مسیرپیمایی واقعی	وابسته به کامپایل

جدول ۲.۱: ویژگی ابزارهای تحلیل ایستای به کاررفته در فاز یک

مزیت اجرای موازی چند ابزار، پوشش بیشتر و کاهش وابستگی به نقاط کور یک ابزار واحد است. عیب آن، تولید هشدارهای همپوشان است؛ به عنوان مثال، ممکن است سه ابزار یک سرریز بافر واحد را با سه شناسه متفاوت گزارش کنند. این هشدارهای تکراری، اگر تفکیک نشوند، هم آمار هشدار نادرست را بدتر نشان می‌دهند و هم بار کاری عامل زبانی را چند برابر می‌کنند. به همین دلیل، در فاز دو، یافته‌های همپوشان با استفاده از اثر انگشت و تطبیق خط در یک گروه ادغام می‌شوند.

۲.۳ تحلیل ایستا-پویا و رویکردهای ترکیبی

تحلیل پویا، برخلاف تحلیل ایستا، برنامه را اجرا می‌کند و رفتار واقعی آن را مشاهده می‌نماید. این رویکرد دقت بالایی برای خطاهایی دارد که در مسیر اجرایی رخ می‌دهند، اما تنها بخش کوچکی از کد را پوشش می‌دهد، زیرا اجرای همه حالت‌های ممکن یک برنامه، به طور کلی ناممکن است. رویکردهای ترکیبی^۳ تلاش می‌کنند این دو مزیت را هم‌زمان به دست آورند؛ برای نمونه، با استفاده از تحلیل ایستا برای یافتن محل‌های مشکوک و سپس اجرای هدفمند برنامه در همان محل‌ها. سامانه پیشنهادی این پژوهش نیز در همین خانواده مفهومی قرار می‌گیرد، با این تفاوت که به جای تحلیل پویا، از یک عامل زبانی به عنوان داور مبتنی بر شواهد استفاده می‌کند.

۲.۴ مدل‌های زبانی بزرگ و عامل‌های نرم‌افزاری

مدل‌های زبانی بزرگ، مدل‌های آماری زبانی با توانایی تولید متن بر اساس پیش‌بینی توکن بعدی هستند که با مقیاس‌دهی حجم داده و پارامتر آموزش داده می‌شوند. این مدل‌ها در چند سال اخیر توانسته‌اند در وظایفی مانند تولید کد، خلاصه‌سازی و تحلیل ایستا مورد استفاده قرار گیرند [۱۱].

عامل نرم‌افزاری^۴ به برنامه‌ای اطلاق می‌شود که با دریافت یک هدف، به طور خودکار برنامه‌ریزی می‌کند، ابزار فراخوانی می‌کند و تا رسیدن به پاسخ، چرخه مشاهده-تصمیم-اقدام را تکرار می‌کند. در این پژوهش، عامل با یک نقش محدود تعریف شده است: داوری یک یافته مشخص بر پایه بسته شواهد ارائه شده. این محدودیت عامدانه است، زیرا عامل باید در برابر وسوسه تولید یافته جدید مقاوم باشد؛ وظیفه عامل یافتن آسیب‌پذیری نیست، بلکه داوری درباره یافته پیشنهادی است.

^۳ hybrid analysis

^۴ Software Agent

۲.۵ استدلال مبتنی بر شواهد

یکی از محدودیت‌های شناخته‌شده مدل‌های زبانی این است که می‌توانند پاسخی روان، مرتبط و در ظاهر مستند تولید کنند، در حالی که هیچ‌یک از اجزای آن پاسخ در متن ورودی موجود نبوده است. این پدیده که ریشه در نبود پیوند میان توکن‌های تولیدشده و جهان واقعی دارد، در ادبیات هوش مصنوعی با مسئله خاک‌افتادگی^۵ توصیف شده است [۱۴].

راهکاری که در این پژوهش به کار گرفته شده، محدودکردن عامل به داوری بر پایه شواهد است. عامل موظف است تنها از محتوای بسته شواهد استفاده کند، در صورت کمبود شاهد، عدم قطعیت را اعلام نماید و به جای استنتاج آزاد، به قطعه کد و ساختار ارائه‌شده استناد کند. این رویکرد با خانواده روش‌های بازیابی‌شده^۶ هم‌خانواده است، با این تفاوت که هدف، افزایش دانش مدل نیست، بلکه مهار آن و کاهش پاسخ‌های بی‌پایه است [۱۳].

۲.۶ مجموعه داده جولیت

مجموعه آزمون جولیت^۷ یک مجموعه داده مرجع برای ارزیابی ابزارهای تحلیل ایستا است که توسط مؤسسه ملی استاندارد و فناوری آمریکا به عنوان بخشی از پایگاه آزمون ارزیابی امنیت نرم‌افزار منتشر شده است [۴]. این مجموعه برای هر نوع آسیب‌پذیری، جفت‌هایی از نسخه آسیب‌پذیر و نسخه سالم از یک سناریوی واحد تولید می‌کند. تنها تفاوت میان نسخه آسیب‌پذیر و سالم، در نقطه‌ای از جریان داده است؛ برای مثال، در یک سناریوی سرریز عدد صحیح، نسخه سالم همان محاسبه را با یک شرط کنترلی همراه می‌کند.

CWE190_Integer_Overflow__int_rand_add_73b_bad.cpp

```
void badSink(list<int> dataList)
{
    int data = dataList.back();
    {
        /* POTENTIAL FLAW: Adding 1 to data could
           cause an overflow */
        int result = data + 1;
        printIntLine(result);
    }
}
```

شکل ۲.۱: نسخه آسیب‌پذیر یک سناریوی جولیت

^۵ Grounding

^۶ Retrieval-Augmented Generation

^۷ Juliet Test Suite

CWE190_Integer_Overflow__int_rand_add_73b_good.cpp

```
void goodB2GSink(list<int> dataList)
{
    int data = dataList.back();
    /* FIX: Add a check to prevent an overflow */
    if (data < INT_MAX) {
        int result = data + 1;
        printIntLine(result);
    }
}
```

شکل ۲.۲: نسخه سالم همان سناریو؛ تنها تفاوت، یک شرط کنترلی است

ویژگی جفت‌بودن نسخه‌ها، هم فرصت و هم تهدید است. فرصت از آن رو که هر دو نسخه زمینه تقریباً یکسانی دارند، بنابراین یک هشدار نادرست را می‌توان به‌سادگی به توضیحی بیرونی نسبت داد. تهدید از آن رو که اگر مرزبندی داده‌ها درست انجام نشود، نسخه‌های مرتبط یک سناریو می‌توانند در دو بخش آموزش و آزمون قرار گیرند و نتیجه ارزیابی به‌شدت خوش‌بینانه شود. به همین دلیل، در این پژوهش برای هر نمونه یک شناسه گروه تعریف شده است تا نمونه‌های هم‌خانواده در یک بخش قرار گیرند.

۲.۷ کارهای پیشین

کارهای پیشین را می‌توان در سه دسته دید. دسته نخست، ارزیابی‌های انتقادی ابزارهای تحلیل ایستا هستند که نشان می‌دهند دقت ظاهری این ابزارها در صورت نبود تعریف دقیق معیارها، گمراه‌کننده است [۷] [۸] و این که هشدارهای نادرست و از دست رفتن آسیب‌پذیری، هر دو فراوانند [۶]. دسته دوم، روش‌های یادگیری ماشین برای تشخیص آسیب‌پذیری هستند که یادگیری عمیق روی کد آسیب‌پذیر و سالم به کار می‌برند و نشان داده‌اند که می‌توانند دقت بالاتری از ابزارهای کلاسیک به دست آورند [۹]؛ با این حال، این روش‌ها معمولاً مبانی نظری روشنی برای توضیح دلیل هر هشدار ارائه نمی‌کنند و در برابر کد ناآشنا به‌سختی تعمیم می‌یابند. دسته سوم، کاربرد مدل‌های زبانی بزرگ در تحلیل کد است؛ از جمله بررسی سهم کد تولیدشده توسط دستیارهای برنامه‌نویسی از نظر آسیب‌پذیری [۱۰] [۱۱] که نشان می‌دهد توانایی تولید کد، به‌تنهایی تضمینی برای ایمنی کد نیست.

دسته	نمونه	نقش	محدودیت	جایگاه این کار
ارزیابی انتقادی	[۶] [۸]	سنجش واقع‌بینانه	وابسته به مجموعه داده	مبانی تعریف معیار
یادگیری ماشین	[۹]	دقت بالاتر	بدون توضیح‌پذیری	مقایسه عددی
مدل زبانی	[۱۰] [۱۱]	فهم زبان طبیعی	توهم و عدم قطعیت	تلفیق با ایستا

جدول ۲.۲: جایگاه کار پیشنهادی در میان کارهای پیشین

۲.۸ جایگاه کار پیشنهادی

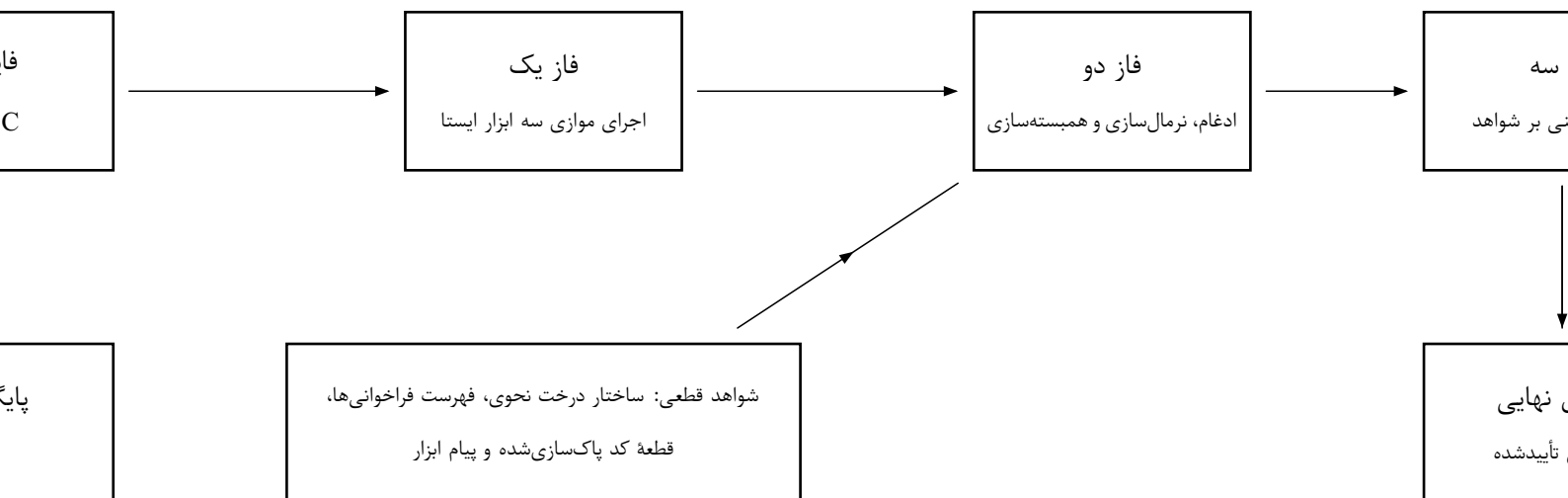
کار پیشنهادی این پژوهش از دسته سوم متأثر است، اما بر یک نکته روش‌شناختی تأکید دارد که در بسیاری از کارهای موجود رعایت نمی‌شود: ارزیابی کمی باید با تعریف صریح معیارها و کنترل نشت برچسب همراه باشد. همچنین، به جای جایگزینی ابزارهای ایستا، این کار آن‌ها را به عنوان تأمین‌کننده نامزد به کار می‌گیرد و از عامل زبانی تنها برای پالایش نامزدها بهره می‌گیرد. این انتخاب، امکان گزارش یک نقطه کاری قابل تنظیم را فراهم می‌کند: سامانه می‌تواند با تغییر آستانه تأیید، روی نقطه‌ای از منحنی مبادله دقت و یادآوری بنشیند که برای هر سازمان مناسب‌تر است.

فصل سوم

طراحی و پیاده‌سازی سامانه

۳.۱ معماری کلی

سامانه پیشنهادی از سه فاز متوالی تشکیل شده است. فاز یک، تحلیل ایستا است و مسئولیت آن تولید فهرستی از یافته‌های خام است. فاز دو، نرمال‌سازی و همبسته‌سازی یافته‌ها بر عهده دارد و آن‌ها را به گروه‌هایی که هر یک نماینده یک نقطه ضعف واحد هستند، تبدیل می‌کند. فاز سه، بازبینی هر گروه توسط عامل زبانی است و تصمیم نهایی را می‌گیرد. این تفکیک فازها عمدی است: هر فاز یک پرسش مشخص را پاسخ می‌دهد و خروجی آن قابل ارزیابی جداگانه است، بنابراین می‌توان اثر هر فاز را مستقل از دو فاز دیگر سنجید.



شکل ۳.۱: معماری سه‌فاز سامانه و جریان داده میان فازها

جداسازی فازها یک مزیت عملی نیز دارد. اجرای ابزارهای ایستا هزینه پردازشی محلی دارد، در حالی که فراخوانی مدل زبانی هزینه زمانی و شبکه‌ای دارد. جدا بودن این دو، امکان اجرای موازی آن‌ها را روی مجموعه‌ای از نخ‌ها و یک زمان‌بند پرتاب^۱ می‌دهد، بدون آنکه فراخوانی‌های زبانی در انتظار، اجرای ابزارهای ایستا را متوقف کنند.

۳.۲ فاز یک: تحلیل ایستا

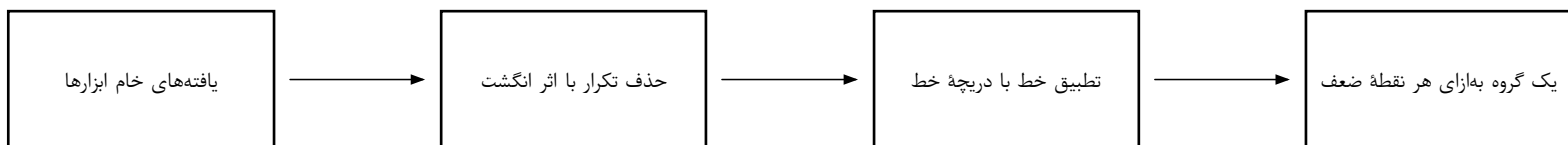
در فاز یک، سه ابزار متن‌باز به صورت موازی روی هر فایل از پروژه اجرا می‌شوند و خروجی خام هر ابزار به فهرستی ساخت‌یافته از یافته‌ها تبدیل می‌شود. برای هر یافته، شناسه ابزار، شماره خط، شدت، پیام و در صورت وجود، شناسه CWE ذخیره می‌شود. انتخاب چند ابزار به جای یک ابزار، به دلیل پوشش مکمل آن‌هاست: هیچ‌یک به تنهایی همه سناریوهای مجموعه جولیت را پوشش نمی‌دهد.

۳.۳ فاز دو: نرمال‌سازی و همبسته‌سازی یافته‌ها

خروجی خام ابزارها سه مشکل دارد که پیش از بازبینی باید برطرف شود. نخست، یافته‌های تکراری است؛ چند ابزار ممکن است یک نقص واحد را گزارش کنند. دوم، گزارش‌های ناهمگون است؛ پیام‌ها قالب یکسانی ندارند و برخی تنها

^۱ Scheduler

یک قاعدهٔ سبک را گزارش می‌کنند. سوم، یافته‌های پراکنده‌اند؛ یک نقطهٔ ضعف واحد ممکن است در چند خط و با چند شناسهٔ متفاوت ظاهر شود.



شکل ۳.۴: زنجیرهٔ ادغام یافته‌های تکراری و تشکیل گروه‌های بازبینی

برای رفع این سه مشکل، سه گام متوالی تعریف شده است. گام نخست، حذف تکرار با استفاده از یک اثر انگشت شامل نام فایل، نوع آسیب‌پذیری و خط است. گام دوم، تطبیق خط با یک دریچهٔ خط است، به این معنا که دو یافته در فاصلهٔ کمتر از پنج خط از یکدیگر، مربوط به یک نقطهٔ ضعف واحد تلقی می‌شوند. گام سوم، ساخت گروه بازبینی است؛ هر گروه، مجموعهٔ یافته‌های ادغام‌شده به‌همراه فهرست ابزارهای گزارش‌دهندهٔ آن است. هدف از این ادغام، کاهش بار کاری عامل زبانی و جلوگیری از چند بار پرسش دربارهٔ یک نقطهٔ ضعف واحد است.

۳.۴ فاز سه: بازبینی مبتنی بر شواهد

در فاز سه، برای هر گروه یافته، یک درخواست به عامل زبانی ارسال می‌شود و عامل باید یکی از سه تصمیم را برگرداند. این سه تصمیم در جدول ۳.۱ تعریف شده‌اند. دو نکته در این طراحی قابل تأمل است. نخست، وجود گزینهٔ نامطمئن، عامل را از دوگزینه‌ای بودن رها می‌کند و اجازه می‌دهد در نبود شاهد کافی، صریحاً عدم قطعیت خود را اعلام نماید؛ خروجی نامطمئن در گزارش نهایی به‌عنوان نیازمند بررسی انسانی حفظ می‌شود. دوم، سامانه به‌جای استفاده از خروجی مدل به‌عنوان یافته، از آن به‌عنوان دآوری استفاده می‌کند؛ یعنی عامل اجازه ندارد یافتهٔ جدیدی بسازد.

خروجی	معنا	اثر بر گزارش
تأییدشده	CONFIRMED	شاهد کافی برای پذیرش یافته
ردشده	REJECTED	شاهد ناسازگار با یافته
نامطمئن	UNCERTAIN	شاهد ناکافی
		نیازمند بررسی انسانی

جدول ۳.۱: سه خروجی ممکن عامل زبانی

۳.۵ بستهٔ شواهد

کیفیت تصمیم عامل، تابع کیفیت شواهدی است که در اختیارش قرار می‌گیرد. بستهٔ شواهد عمدتاً چهار بخش دارد. بخش نخست، گروه یافته است که شامل خط، شدت، شناسهٔ CWE و ابزارهای گزارش‌دهنده است. بخش دوم، فهرست اعضای گروه با پیام کامل هر ابزار است. بخش سوم، قطعهٔ کد پیرامون خط مورد نظر با شمارهٔ خط است. بخش چهارم که اختیاری است، شواهد ساختاری شامل فهرست توابع، فراخوانی‌ها و واردسازی‌های فایل است.

evidence-packet.json

```
{
  "language": "cpp",
  "group": {
    "id": "G-3",
    "line": 33,
    "severity": "HIGH",
    "cwe": ["CWE-122"],
    "tools": ["cppcheck", "clang"]
  },
  "members": [
    { "tool": "cppcheck", "message": "buffer overflow" },
    { "tool": "clang", "message": "out of bound array access" }
  ],
  "source_context": {
    "start_line": 27,
    "end_line": 39,
    "snippet": " 27:  twoIntsStruct * data;\n ..."
  },
  "structural": {
    "functions": [ { "name": "sym_1902e3", "line": 25 } ],
    "calls": [ { "callee": "action", "line": 33 } ]
  }
}
```

شکل ۳.۲: ساختار بسته شواهد ارسال شده به عامل زبانی

قطعه کد با شعاع هشت خط پیش و پس از خط مورد نظر ساخته می‌شود. این مقدار، مصالحه‌ای است میان هزینه توکن و کفایت زمینه: شعاع کمتر، تعریف متغیر یا نقطه کنترلی مرتبط را از قطعه بیرون می‌گذارد و شعاع بیشتر، توجه مدل را پراکنده می‌کند. شواهد ساختاری با استفاده از تجزیه‌گر درخت نحوی تولید می‌شود و به عامل امکان می‌دهد به جای حدس زدن، فهرست فراخوانی‌های همان تابع را ببیند. در فصل چهارم نشان داده می‌شود که افزودن این بخش، تعداد تأییدهای عامل را افزایش می‌دهد.

۳.۶ پاک‌سازی کد و جلوگیری از نشت برچسب

مجموعه داده جولیت برچسب واقعی هر فایل را درون خود متن فایل قرار می‌دهد. برای نمونه، در فایل‌های آسیب‌پذیر، حاشیه‌نویسی‌هایی با عبارت‌هایی نظیر نشانگر وجود نقص و شماره CWE وجود دارد و نام فایل و نام اشیا نیز شامل واژه

بد یا خوب هستند. اگر این متن عیناً به مدل داده شود، مدل می‌تواند بدون هیچ تحلیلی پاسخ درست را از روی نام فایل حدس بزند و نتیجه آزمایش، اندازه‌گیری توانایی تحلیل نخواهد بود، بلکه اندازه‌گیری توانایی خواندن برچسب. برای جلوگیری از این نشت، سه اقدام متوالی در مازول پاک‌سازی انجام می‌شود. نخست، محتوای حاشیه‌نویسی‌ها در جای خود با فاصله جایگزین می‌شود، به گونه‌ای که تعداد خط‌ها و شماره خط گزارش شده توسط ابزارها تغییر نکند. دوم، شناسه‌های افشاگر، مانند نام‌هایی که شامل واژه بد یا خوب هستند، به نام‌هایی پایدار و بی‌معنا نگاشت می‌شوند؛ به این ترتیب که یک نام در قطعه کد، در شواهد ساختاری و در پیام ابزار، همیشه به یک شکل ظاهر شود. سوم، پاک‌سازی روی تمام فیلدهای متنی بسته شواهد اعمال می‌شود، نه فقط بر قطعه کد، زیرا پیام ابزارها و نام فایل نیز می‌توانند برچسب را افشا کنند.

CWE122_Heap_Based_Buffer_Overflow__c_CWE805_struct_memcpy_82a_bad.cpp

```

24:
25: void sym_1902e3()
26: {
27:     twoIntsStruct * data;
28:     data = NULL;
29:
30:     data = (twoIntsStruct *)malloc(50*sizeof(twoIntsStruct));
31:     if (data == NULL) {exit(-1);}
32:     sym_9623a0* base0bject = new sym_5b7d8f;
33:     base0bject->action(data);
34:     delete base0bject;
35: }
```

شکل ۳.۳: قطعه کد پس از پاک‌سازی؛ حاشیه‌نویسی‌ها محو و شناسه‌های افشاگر بازنام‌گذاری شده‌اند

یک استثنا عمدی است: ارجاع‌هایی که شکل CWE-122 دارند بازنویسی نمی‌شوند، زیرا سازوکار تطبیق یافته‌ها به آن‌ها نیاز دارد. همچنین، برای اطمینان از پایداری رفتار سامانه، آزمون‌های خودکاری نوشته شده است که در صورت بازگشت نشت برچسب، آزمایش شکست می‌خورد؛ زیرا در آن صورت همه آزمایش‌های مبتنی بر مدل زبانی به‌طور خاموش به سنجش نشت تبدیل می‌شوند.

۳.۷ سیاست تطبیق یافته‌ها و معیارها

سنجش خروجی سامانه، نیازمند تصمیم درباره این است که یک هشدار چه زمانی با یک آسیب‌پذیری واقعی منطبق دانسته شود. در این سامانه، سه قانون به کار می‌رود که در جدول ۳.۲ خلاصه شده‌اند.

قانون تطبیق	شرط	دلیل
هم‌بودن فایل	نام فایل یکسان	تطبیق در سطح پروژه واقعی
خانواده CWE	هم‌پوشانی خانواده	ابزارها CWE والد گزارش می‌کنند
فاصله خط	کمتر از ۵ خط	خطای گزارش ابزار جبران می‌شود

جدول ۳.۲: سیاست تطبیق یافته‌ها با برچسب واقعی

نخستین قانون، الزام هم‌بودن فایل است؛ بدون آن، هیچ تطبیقی پذیرفته نمی‌شود. دومین قانون، استفاده از خانواده CWE است. این قانون از یک واقعیت عملی ناشی می‌شود: ابزارها اغلب شناسه والد را گزارش می‌کنند، در حالی که برچسب مجموعه داده فرزند را نام می‌برد؛ برای نمونه، یک ابزار ممکن است سرریز بافر کلاسیک را گزارش کند، در حالی که نمونه متعلق به سرریز بافر مبتنی بر حافظه چرخه‌ای است. بدون نگاشت خانواده، این یافته‌های واقعی، هم‌زمان هشدار نادرست و از دست رفتن آسیب‌پذیری شمرده می‌شوند. سومین قانون، تحمل پنج خطی فاصله خط است.

در مورد معیارها، یک تصمیم روش‌شناختی مهم گرفته شده است. مثبت درست، مثبت نادرست و منفی نادرست در دانه‌بندی یافته محاسبه می‌شوند، یعنی هر هشدار جداگانه شمرده می‌شود. منفی درست اما در دانه‌بندی فایل محاسبه می‌شود، یعنی یک فایل سالم تنها زمانی منفی درست است که هیچ هشدار نداشت باشد. دلیل این انتخاب آن است که هشدار نادرست، پدیده‌ای در سطح فایل است: توسعه‌دهنده با فایلی روبه‌رو می‌شود که هشدار دارد، نه با یک خط منفرد. برای رفع ابهام، نرخ هشدار روی فایل سالم به‌طور جداگانه گزارش می‌شود.

۳.۸ زیرساخت اجرا و تکرارپذیری

یکی از پیش‌نیازهای هر ادعای کمی، تکرارپذیری است. به همین دلیل، هر اجرای سامانه، در کنار نتایج، فراداده‌ای شامل نسخه هر ابزار، نام و نسخه مدل زبانی، نشانی سرویس، شعاع قطعه کد، تحمل خط، سیاست تطبیق CWE و مدت اجرا را در کنار نتایج ذخیره می‌کند. همچنین، اجرای بازبینی به‌گونه‌ای پیاده‌سازی شده است که در صورت قطع پاسخ سرویس، اجرای دوباره تنها موارد ناموفق را پردازش کند. این ویژگی، هم خطای شبکه را از خطای روش‌شناختی جدا می‌کند و هم هزینه ارزیابی تکراری را کاهش می‌دهد.

فصل چهارم

ارزیابی تجربی

۴.۱ پرسش‌های پژوهش

ارزیابی تجربی این پژوهش بر چهار پرسش استوار است. پرسش نخست: بازبینی مبتنی بر شواهد، آیا هشدارهای نادرست تحلیل ایستا را کاهش می‌دهد؟ پرسش دوم: بهای این کاهش از نظر یادآوری چقدر است؟ پرسش سوم: افزودن شواهد ساختاری، اثر بازبینی را بهبود می‌دهد؟ پرسش چهارم: در مقایسه با هر یک از دو مؤلفه به‌تنهایی، ترکیب آن‌ها چه تصویری از منحنی مبادله دقت و یادآوری به دست می‌دهد؟

۴.۲ مجموعه داده و آماده‌سازی آن

برای آنکه هر چهار پیکربندی روی یک جمعیت یکسان سنجیده شوند، از مجموعه داده جولیت یک زیرمجموعه متعادل و لایه‌بندی شده ساخته شد. این زیرمجموعه شامل ۶۰۰ فایل است: ۳۰۰ فایل آسیب‌پذیر و ۳۰۰ فایل سالم. لایه‌بندی بر پایه نوع آسیب‌پذیری و برچسب انجام شده است، به‌گونه‌ای که از هر یک از چهار لایه، یعنی ترکیب نوع آسیب‌پذیری با برچسب، دقیقاً ۱۵۰ نمونه با بذر ثابت انتخاب شود.

ویژگی	مقدار	توضیح
زبان‌ها	C, C++	هر دو خانواده کد مدل می‌شوند
آسیب‌پذیری‌ها	CWE-122, CWE-190	سرریز بافر و سرریز عدد صحیح
تعداد نمونه ارزیابی	۶۰۰	۳۰۰ آسیب‌پذیر و ۳۰۰ سالم
لایه‌بندی	۱۵۰ در هر لایه	چهار لایه، با بذر ۴۲
ابزارهای فاز یک	cppcheck 2.22, flawfinder 2.0.20, clang 22	نسخه‌های به‌کاررفته
مدل بازبین	mimo-v2.6-flash-free	از راه سرور محلی

جدول ۴.۱: مشخصات مجموعه داده ارزیابی و پیکربندی آزمایش

علت استفاده از زیرمجموعه به‌جای کل مجموعه داده، دوگانه بود. از یک سو، مقایسه چهار پیکربندی نیازمند جمعیت یکسان است. از سوی دیگر، آزمایش چهارم، یعنی بازبینی عامل زبانی روی تک‌تک فایل‌ها، پرهزینه‌ترین بخش کار است؛ هزینه آن تقریباً با تعداد فایل‌ها متناسب است. انتخاب زیرمجموعه لایه‌بندی شده، امکان مقایسه معتبر و در عین حال اجرایی شدن آزمایش‌ها را فراهم کرد. برای اطمینان از این‌که نتایج فاز یک وابسته به اندازه مجموعه نباشد، آزمایش فاز یک علاوه بر زیرمجموعه، روی کل ۴۰۹۸ فایل نیز اجرا شد که نتایج آن در بخش ۴.۸ گزارش می‌شود.

۴.۳ طراحی آزمایش‌ها

برای پاسخ به پرسش‌های پژوهش، چهار پیکربندی روی همان زیرمجموعه اجرا شد. این طراحی، هر یک از سه مؤلفه سامانه را در کنار دیگری و در کنار ترکیب کامل می‌سنجد. پیکربندی نخست، تنها فاز یک را اجرا می‌کند و همه یافته‌های

ابزارها را بدون هیچ فیلتری به عنوان خروجی سامانه در نظر می‌گیرد؛ این پیکربندی خط پایه ابزارهای تحلیل ایستا را تعریف می‌کند و در ادامه A نامیده می‌شود. پیکربندی دوم، فاز یک را حذف می‌کند و هر فایل را به طور کامل در اختیار عامل زبانی قرار می‌دهد؛ این پیکربندی توان عامل زبانی را بدون کمک ابزارهای ایستا می‌سنجد و B نامیده می‌شود. پیکربندی سوم، خروجی ابزارها را در اختیار عامل می‌گذارد، اما بدون شواهد ساختاری، و C نامیده می‌شود. پیکربندی چهارم، شواهد ساختاری را نیز می‌افزاید و D نامیده می‌شود. دو پیکربندی پایانی دقیقاً یکسان‌اند، جز در وجود یا نبود شواهد ساختاری، و همین تمایز، اثر آن شواهد را به طور جداگانه قابل اندازه‌گیری می‌کند.

۴.۴ معیارها و سیاست تطبیق

همان‌گونه که در فصل سوم بیان شد، سه معیار در دانه‌بندی یافته و یک معیار در دانه‌بندی فایل محاسبه می‌شود. دقت و یادآوری بر پایه انطباق یافته‌ها با برچسب واقعی و با سیاست تطبیق توصیف‌شده به دست می‌آید و نرخ هشدار روی فایل سالم، سهم فایل‌های سالمی را نشان می‌دهد که دست‌کم یک هشدار دریافت کرده‌اند. گزارش این معیارها به همراه ماتریس سردرگمی، امکان بازسازی کامل هر عدد را فراهم می‌کند.

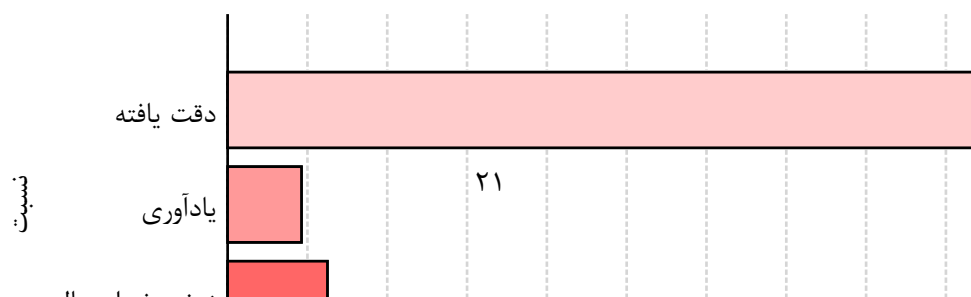
۴.۵ نتایج اصلی

نتایج چهار پیکربندی در جدول ۴.۲ آمده است. نخستین نکته قابل توجه، این است که هیچ پیکربندی برتری مطلق ندارد و هر یک در نقطه‌ای متفاوت از منحنی مبادله دقت و یادآوری می‌نشیند.

آزمایش	پیکربندی	مثبت درست	مثبت نادرست	دقت	یادآوری	F1	نرخ هشدار روی فایل سالم
A	فقط ایستا	۱۱	۱۸	۰٫۳۷۹	۰٫۳۷	۰٫۰۶۷	۰٫۰۵۰
B	فقط عامل زبانی	۴۴	۷۵	۰٫۳۷۰	۰٫۱۴۷	۰٫۲۱۰	۰٫۱۹۷
C	ایستا + عامل	۳	۲	۰٫۶۰۰	۰٫۰۱۰	۰٫۰۲۰	۰٫۰۰۷
D	ایستا + ساختار + عامل	۵	۶	۰٫۴۵۵	۰٫۰۱۷	۰٫۰۳۲	۰٫۰۲۰

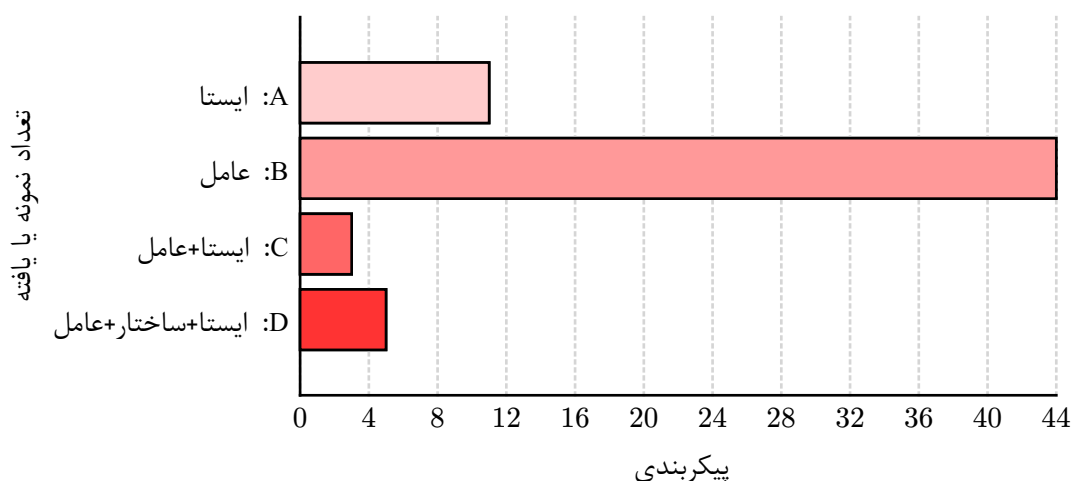
جدول ۴.۲: نتایج اصلی چهار آزمایش روی ۶۰۰ نمونه متعادل

پیکربندی B، یعنی عامل زبانی به تنهایی، بالاترین یادآوری را با ۰٫۱۴۷ به دست می‌آورد؛ چهار برابر یادآوری تحلیل ایستا. این یعنی عامل زبانی چیزهایی می‌بیند که ابزارهای ایستا از آن‌ها غافل‌اند. بهای این افزایش یادآوری، بالا رفتن نرخ هشدار روی فایل‌های سالم به ۰٫۱۹۷ است، یعنی تقریباً یک فایل سالم از هر پنج فایل، هشدار می‌گیرد. دقت یافته‌ها در این پیکربندی تقریباً با پیکربندی A برابر است، بنابراین افزایش یادآوری با کاهش کیفیت یافته‌ها همراه نبوده است.



شکل ۴.۱: مقایسه دقت، یادآوری و نرخ هشدار روی فایل سالم در چهار پیکربندی

پیکربندی A، یعنی تحلیل ایستا به تنهایی، کمترین نرخ هشدار روی فایل سالم را با ۰/۰۵۰ دارد، اما تنها ۰/۰۳۷ از فایل‌های آسیب‌پذیر را کشف می‌کند. دلیل این یادآوری بسیار پایین، آن است که ابزارهای ایستا با قواعد مبتنی بر الگو، تنها بخش کوچکی از سناریوهای جولیت را فعال می‌کنند. این نتیجه، یک محدودیت مهم را آشکار می‌کند: هر چه سامانه بر پایه تحلیل ایستا بنا شود، یادآوری آن سقفی دارد که تعیین آن، از توان واقعی داوری سامانه مستقل است.



شکل ۴.۲: ماتریس سردرگمی چهار پیکربندی در دانه‌بندی فایل و یافته

پاسخ پرسش نخست پژوهش مثبت است و پاسخ پرسش دوم نیز روشن. بازبینی مبتنی بر شواهد، نرخ هشدار روی فایل سالم را از ۰/۰۵۰ در پیکربندی A به ۰/۰۰۷ در پیکربندی C رساند؛ یعنی حدود ۸۷ درصد کاهش. دقت یافته‌ها نیز از ۰/۳۷۹ به ۰/۶۰۰ افزایش یافت. اما بهای این کاهش، افت یادآوری از ۰/۰۳۷ به ۰/۰۱۰ است؛ از یازده مثبت درست در پیکربندی A، تنها سه مورد در پیکربندی C باقی ماند. به بیان دیگر، عامل زبانی علاوه بر نادیده گرفتن هشدارهای نادرست، هشت مورد از یافته‌های درست ابزارها را نیز کنار گذاشت. بنابراین سامانه یک کاهش هشدار نادرست به دست می‌آورد، اما این کاهش، کامل و بدون هزینه نیست.

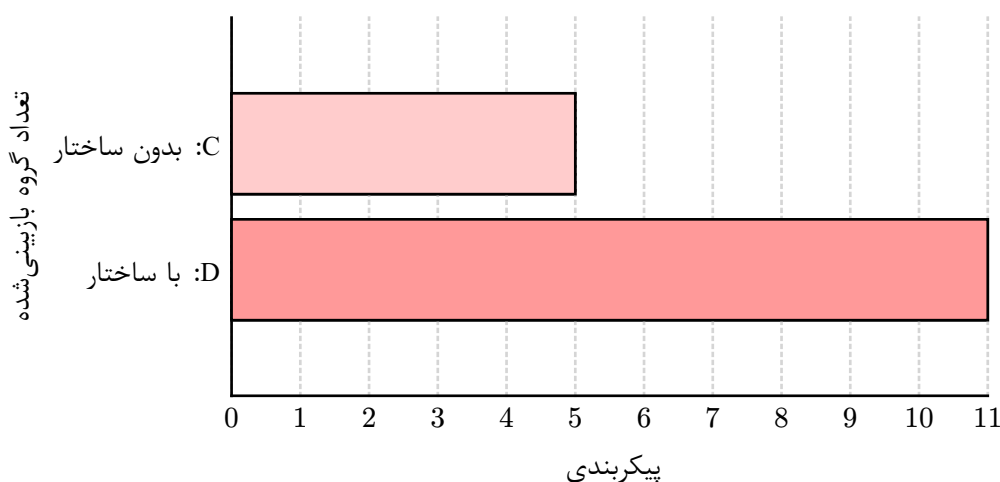
۴.۶ تحلیل قیف تصمیم

برای فهمیدن دلیل افت یادآوری، باید به تصمیم‌های خود عامل نگاه کرد، نه فقط به معیارهای تجمیعی. قیف تصمیم در جدول ۴.۳ آمده است.

آزمایش	تعداد گروه بازیینی	تأییدشده	ردشده / نامطمئن
B	۶۰۰ فایل	۱۱۹	۴ / ۴۷۷
C	۳۱	۵	۱۳ / ۱۳
D	۳۱	۱۱	۶ / ۱۴

جدول ۴.۳: کیف تصمیم عامل زبانی در هر پیکربندی

در پیکربندی C، عامل از ۳۱ گروه، تنها ۵ گروه را تأیید کرد و ۱۳ گروه را رد کرد. بخش قابل توجه، سهم بالای خروجی نامطمئن است؛ ۱۳ گروه از ۳۱ گروه، یعنی حدود ۴۲ درصد، با حالت نامطمئن بسته شدند. این رفتار از آن جهت مطلوب است که عامل به جای داوری قطعی، کمبود شاهد را صادقانه اعلام می‌کند. اما از آن جهت که در خروجی نهایی تنها یافته‌های تأییدشده شمرده می‌شوند، این رفتار به‌طور خودکار به‌معنای از دست رفتن یافته درست است.



شکل ۴.۳: اثر شواهد ساختاری بر تصمیم‌های عامل زبانی

مقایسه دو پیکربندی C و D اثر شواهد ساختاری را روشن می‌کند. با افزودن شواهد ساختاری، تعداد تأییدها از ۵ به ۱۱ رسید و تعداد خروجی‌های نامطمئن از ۱۳ به ۶ کاهش یافت. مثبت‌های درست نیز از ۳ به ۵ افزایش یافت و یادآوری از ۰/۰۱۰ به ۰/۰۱۷ رسید. این نتیجه، پاسخ مثبت پرسش سوم است: شواهد ساختاری، توان عامل را در تصمیم‌گیری افزایش می‌دهد و بخشی از یافته‌هایی را که بدون آن نامطمئن می‌ماند، به تأیید می‌رساند. با این حال، بهای این افزایش، رشد مثبت‌های نادرست از ۲ به ۶ و افزایش نرخ هشدار روی فایل سالم از ۰/۰۰۷ به ۰/۰۲۰ است. شواهد بیشتر، عامل را مطمئن‌تر می‌کند و اطمینان بیشتر در این سامانه، هم‌زمان هم یافته درست و هم یافته نادرست بیشتری تولید می‌کند.

۴.۷ نتایج به تفکیک نوع آسیب‌پذیری

تفکیک نتایج به تفکیک نوع آسیب‌پذیری، تصویری متفاوت و آموزنده ارائه می‌دهد.

آزمایش	نوع	مثبت درست	دقت	یادآوری	F1
A	CWE-122	۱۱	۰/۵۵۰	۰/۰۷۳	۰/۱۲۹
A	CWE-190	۰	۰/۰۰۰	۰/۰۰۰	۰/۰۰۰
B	CWE-122	۳۵	۰/۵۱۵	۰/۲۳۳	۰/۳۲۱
B	CWE-190	۹	۰/۳۳۳	۰/۰۶۰	۰/۱۰۲
C	CWE-122	۳	۰/۶۰۰	۰/۰۲۰	۰/۰۳۹
C	CWE-190	۰	۰/۰۰۰	۰/۰۰۰	۰/۰۰۰
D	CWE-122	۵	۰/۴۵۵	۰/۰۳۳	۰/۰۶۲
D	CWE-190	۰	۰/۰۰۰	۰/۰۰۰	۰/۰۰۰

جدول ۴.۴: نتایج به تفکیک نوع آسیب‌پذیری

مهم‌ترین یافته این بخش آن است که هیچ‌یک از دو پیکربندی مبتنی بر تحلیل ایستا، حتی یک نمونه از ۱۵۰ نمونه سرریز عدد صحیح را کشف نکردند. برای این نوع آسیب‌پذیری، یادآوری هر سه پیکربندی C، A و D دقیقاً صفر است. تنها پیکربندی B توانسته است در این نوع، با یادآوری ۰/۰۶۰ و دقت ۰/۳۳۳، چند نمونه را کشف کند. این تفاوت، ریشه در ماهیت دو نوع آسیب‌پذیری دارد. آسیب‌پذیری سرریز بافر، الگوی شناخته‌شده‌ای دارد و قواعد الگویی ابزارها آن را فعال می‌کنند، در حالی که سرریز عدد صحیح خود یک شکست بنیادی در مدل نوع صحیح را آشکار می‌کند و قاعده‌ای که آن را بگیرد، بسیار دشوارتر است. نتیجه عملی این یافته آن است که ابزارهای ایستا برای پوشش این دسته از آسیب‌پذیری‌ها به‌تنهایی کافی نیستند و سامانه‌های ترکیبی که در کنار تحلیل ایستا، فایل را به‌طور کامل به مدل زبانی می‌سپارند، پوشش به‌مراتب گسترده‌تری به دست می‌آورند؛ به بهای نرخ بالاتر هشدار روی فایل‌های سالم.

۴.۸ بررسی در مقیاس کامل

برای بررسی وابستگی نتایج فاز یک به اندازه مجموعه، پیکربندی A علاوه بر زیرمجموعه ۶۰۰ نمونه‌ای، روی کل ۴۰۹۸ فایل مجموعه‌داده نیز اجرا شد. نتیجه با مقدار زیرمجموعه هم‌خوان است: دقت ۰/۴۱۲، یادآوری ۰/۰۴۶ و نرخ هشدار روی فایل سالم ۰/۰۵۱. بنابراین محدودیت یادآوری تحلیل ایستا، ویژگی ابزارها و مجموعه‌داده است، نه اثر تصادفی نمونه‌گیری.

۴.۹ تهدیدهای روایی

در تفسیر نتایج باید به چند محدودیت توجه کرد. نخست، اعداد مثبت درست در پیکربندی‌های C و D کوچک‌اند؛ با سه تا پنج مثبت درست، هر تفاوت کوچکی می‌تواند نرخ‌ها را به‌شدت جابه‌جا کند و این اعداد باید با احتیاط تفسیر شوند. دوم، هر چهار پیکربندی روی یک مدل زبانی مشخص ارزیابی شده‌اند و تعمیم نتایج به سایر مدل‌ها نیازمند آزمایش

تکمیلی است. سوم، مجموعه داده جولیت واقعی نیست؛ کد آن قالب محور و کوتاه است و همین امر، بازبینی را آسان تر از کد واقعی و بزرگ می کند. چهارم، چون هر نسخه سالم جولیت با نسخه آسیب پذیر هم سناریو متناظر است، یک بازبینی که فقط نقطه تفاوت را می بیند، ممکن است به طور مصنوعی خوب عمل کند؛ به همین دلیل، در پیکربندی C و D تنها قطعه کد پیرامون محل هشدار به عامل داده می شود، نه مقایسه با نسخه متناظر.

در پایان، نتیجه کلی آزمایش ها را می توان در یک جمله خلاصه کرد: عامل زبانی، هشدارهای نادرست تحلیل ایستا را به طور چشمگیری کاهش می دهد، اما این کاهش با افت محسوس یادآوری همراه است، و افزودن شواهد ساختاری این مبادله را به نقطه کاری متعادل تری می رساند.

فصل پنجم

نتیجه‌گیری و کارهای آینده

۵.۱ جمع‌بندی

در این پژوهش، سامانه‌ای سه‌فاز برای کاهش هشدارهای نادرست تحلیل ایستا به کمک بازبینی مبتنی بر شواهد با عامل زبانی طراحی، پیاده‌سازی و ارزیابی شد. خروجی اصلی این سامانه، گزارشی است که تنها یافته‌های دارای شاهد کافی را در بر می‌گیرد و در کنار هر یافته، دلیل و شواهد آن را نیز ارائه می‌کند. مهم‌ترین یافته تجربی این پژوهش آن است که بازبینی مبتنی بر شواهد، نرخ هشدار روی فایل‌های سالم را از ۰/۰۵۰ به ۰/۰۰۷ رساند؛ یعنی حدود ۸۷ درصد کاهش، و دقت یافته‌ها را از ۰/۳۷۹ به ۰/۶۰۰ افزایش داد. این کاهش اما رایگان نبود: یادآوری از ۰/۰۳۷ به ۰/۰۱۰ افت کرد. عامل زبانی، بخشی از یافته‌های درست ابزارهای ایستا را نیز به دلیل کمبود شاهد کنار گذاشت.

دو نتیجه دیگر، تصویر کامل‌تری از توان این رویکرد به دست می‌دهند. نخست، تحلیل ایستا به‌تنهایی سقف یادآوری پایینی دارد و این سقف، از توان داوری سامانه مستقل است. دوم، عامل زبانی به‌تنهایی، با یادآوری ۰/۱۴۷، چهار برابر تحلیل ایستا آسیب‌پذیری یافت، اما نرخ هشدار روی فایل سالم آن به ۰/۱۹۷ رسید. تنها پیکربندی‌ای که به هر دو نوع آسیب‌پذیری مجموعه‌داده پوشش داد، پیکربندی B بود؛ به‌ویژه برای سرریز عدد صحیح که هیچ‌یک از پیکربندی‌های مبتنی بر تحلیل ایستا حتی یک نمونه از ۱۵۰ نمونه را کشف نکردند.

۵.۲ محدودیت‌ها

محدودیت‌های این پژوهش را می‌توان در چهار دسته دید. نخست، محدودیت اندازه نمونه؛ تعداد یافته‌های درست در پیکربندی‌های ترکیبی کوچک است و نرخ‌های گزارش‌شده حساس‌اند. دوم، محدودیت مجموعه‌داده؛ کد جولیت قالب‌محور و کوتاه است و یافته‌های آن لزوماً به کد واقعی و بزرگ تعمیم نمی‌یابد. سوم، محدودیت مدل؛ ارزیابی تنها روی یک مدل زبانی انجام شده است. چهارم، محدودیت سیاست تطبیق؛ با توجه به آنکه برچسب خط در مجموعه‌داده ثبت نشده است، تطبیق بر پایه فایل و خانواده CWE انجام می‌شود و این سیاست، تا حدی سخاوتمندانه است.

۵.۳ کارهای آینده

- بر پایه نتایج این پژوهش، مسیرهای زیر برای ادامه کار پیشنهاد می‌شود.
 - بازطراحی بسته شواهد به‌گونه‌ای که به‌جای قطعه‌ای از کد، مسیر داده میان منبع و نقطه خطر را در اختیار عامل قرار دهد. انتظار می‌رود این تغییر، مهم‌ترین گلوگاه فعلی، یعنی کاهش هم‌زمان یادآوری و هشدار نادرست، را برطرف کند.
 - استفاده از خروجی نامطمئن به‌جای دور انداختن آن. از آنجا که ۴۲ درصد گروه‌ها در پیکربندی C با حالت نامطمئن بسته شدند، رتبه‌بندی این موارد و ارائه آن‌ها در یک صفحه جداگانه، می‌تواند بخشی از یادآوری از دست‌رفته را بدون افزودن هشدار نادرست بازبازی کند.

- تنظیم نقطه کاری سامانه بر پایه هزینه نسبی خطا. با آستانه متفاوت برای تأیید، می‌توان سامانه را برای محیط‌هایی که هشدار نادرست پرهزینه‌تر است، یا محیط‌هایی که از دست رفتن آسیب‌پذیری پرهزینه‌تر است، تنظیم کرد.
- ارزیابی روی مجموعه داده‌های دیگر و کد واقعی. تکرار آزمایش‌ها روی مجموعه داده‌هایی با کد بلندتر و وابستگی بین‌فایلی، مشخص می‌کند که نتایج تا چه اندازه به ویژگی جolیت وابسته‌اند.
- توسعه بازبینی تک‌مرحله‌ای به یک عامل چندمرحله‌ای با ابزار پرس‌وجو. اگر عامل بتواند به‌جای اتکا به قطعه ثابت کد، خود زمینه لازم را از فایل و ساختار پروژه استخراج کند، انتظار می‌رود نرخ خروجی نامطمئن کاهش یابد.

فهرست مراجع

- [١] cppcheck, "A tool for static C/C++ code analysis." [Online]. Available: <https://cppcheck.sourceforge.io/>
- [٢] flawfinder, "A free, open source C/C++ program analyzer." [Online]. Available: <https://sourceforge.net/projects/flawfinder/>
- [٣] The clang static analyzer, "A static analyzer for C/C++ built on top of the LLVM compiler infrastructure." [Online]. Available: <https://clang-analyzer.llvm.org/>
- [٤] National Institute of Standards and Technology, "Juliet C/C++ Test Suite," in Software Assurance and Vulnerability Metrics, SARD test suites. [Online]. Available: <https://samate.nist.gov/SARD/test-suites/112>
- [٥] MITRE, "CWE-122: Heap-based Buffer Overflow." [Online]. Available: <https://cwe.mitre.org/data/definitions/122.html>
- [٦] S. B. G. Ayewah, D. M. Ayewah, and W. E. Jones, "What do software developers know about security? A survey," in Proc. IEEE Symp. Security and Privacy, May 2018, pp. 754-769.
- [٧] M. Arnold, S. Crnogorac, and O. Vitek, "A study of false negatives in compiler and static analysis tools," in Proc. ACM SIGPLAN Conf. Companion on Object-Oriented Programming, Systems, Languages, and Applications, 2014.
- [٨] R. Love, R. Lam, S. Habra, and M. Zyp, "An empirical study of false negatives in C/C++ static analyzers," in Proc. Int. Workshop on Programming Languages in Engineering, 2014, pp. 76-86.
- [٩] M. Sridharan, S. Das, I. Cen, L. Wang, and D. Black, "Summit: Scaling deep learning for vulnerability detection," in Proc. ACM SIGSAC Conf. Computer and Communications Security, 2020, art. no. 2151.
- [١٠] H. Pearce, B. Tan, S. Sastry, D. Drozhdzhina, and K. Pernice, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in Proc. IEEE Symp. Security and Privacy, 2022, pp. 1861-1871.

- [۱۱] A. Wang et al., “Large language models for software engineering: Survey and open problems,” 2024. [Online]. Available: <https://arxiv.org/abs/2310.03533>
- [۱۲] S. Yao et al., “Tree-sitter: An incremental parsing system,” 2021. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>
- [۱۳] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459-9474.
- [۱۴] E. M. Bender et al., “On the dangers of stochastic parrots: Can language models be too big?,” in *Proc. ACM Fairness, Accountability, and Transparency*, 2021, pp. 610-623.
- [۱۵] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do users write more insecure code with AI assistants?,” in *Proc. ACM Conf. Computer Communications Security*, 2023, art. no. 754.
- [۱۶] B. Lippmann, “Analyzing the security of program code,” in *Proc. ACM Int. Conf. Genetic and Evolutionary Computation*, 2011, pp. 655-660.